ScheduleFlow User Guide

ScheduleFlow is a local command line application for university students balancing coursework deadlines, project work, classes and other fixed events. Add tasks with remaining-work estimates, record your progress, and reserve time for weekly or one-time commitments. Generate a study plan to find time for your work before its deadlines and identify work that cannot fit.

Study hours default to **08:00–22:00 every day**, including weekends. You can set recurring weekday hours and override individual dates. Work is scheduled in **30-minute slots**, and any work that does not fit is reported explicitly.

> **Guide scope:** This guide describes the current development build for the upcoming v2.0 release. Earlier published JARs may have fewer commands. Successful changes save automatically, and IDs and generated plans survive restarts.

Use blocking multiple dates to reserve a continuous interval or the same daily window across several dates. Use one-time commitments for one date.

Contents

- Quick start
- Command format
- Tasks
  - Recording duration (tasks and commitments)
  - Recording progress
  - Adding a task
  - Listing tasks
  - Editing a task
  - Weekly recurring tasks
  - Completing and viewing completed tasks
  - Reopening a completed task
  - Deleting a task
- Commitments
  - Adding a commitment
  - Adding a one-time commitment
  - Editing a commitment
  - Skipping one occurrence
  - Listing commitments
  - Deleting a commitment
- Study hours
- Study plans
  - Generating a plan
  - Marking a study period unavailable
  - Blocking multiple dates
  - Session limits and breaks

    - Planning hours and date range
    - Unallocated work
- Viewing the schedule
    - Schedule labels
    - Worked schedule example
    - Planning partway through the day
- Help and exit
    - Viewing help
    - Exiting the program
- Saving and restoring data
- Troubleshooting and FAQ
    - What if my work does not fit before its deadline?
    - How do I mark work as complete or correct an entry?
    - Why did my schedule disappear?
    - Can I change the study hours or slot size?
    - Can I schedule more than a year ahead?
    - Why was my command rejected?
    - What should I do if the application cannot load my data?
- Command summary

Quick start

1. Install **Java 25** and check it in a terminal:

    ```
    java -version
    ```

2. Place `scheduleflow.jar` in a dedicated folder. This guide targets the upcoming v2.0 development build; use its matching candidate JAR for these commands. The older published v1.0 JAR has fewer features. A developer can build the candidate using the Developer Guide.
3. Open a terminal **in that folder** and launch:

    ```
    java -ea -jar scheduleflow.jar
    ```

    `-ea` enables internal assertions; distribution launchers enable them automatically. Supply no additional application arguments. The folder you launch from determines which saved data is used. An absent file starts an empty session at `scheduleflow>`.
4. Type `help` and press Enter. Do not type the prompt itself. Then try:

    ```
    task add n/CS2113 draft due/2026-11-02 1800 d/45
    commitment add n/CS2113 lecture day/MON start/1000 d/120
    plan
    schedule today
    ```

    Replace the deadline with a future date within the allowed range before entering it. Allocation depends on the current time and available study hours; partial plans report work that did not fit.

5. Enter `exit`. Successful changes and the generated plan are already saved. Relaunch from the same folder and use `task list`, `commitment list` and `schedule today` to view them. After a date change, use `schedule YYYY-MM-DD` for the date you originally viewed. Viewing a saved schedule does not move its sessions forward.

Command format

Enter each command on one line and press Enter. Terminal transcripts include the `scheduleflow>` prompt; copy only the command after it.

In command formats:

- Uppercase placeholders such as `NAME` and `MINUTES` are values you replace. Keep literal keywords such as `WEEKLY`, weekday codes such as `MON`, and ID prefixes such as `T` in the case shown.
- Square brackets mark optional fields; do not type the brackets. A group such as `[from/YYYY-MM-DD until/YYYY-MM-DD]` means supply both fields or omit both.
- A vertical bar (`|`) separates alternatives; choose one and do not type the bar. For example, `hours DAY|YYYY-MM-DD` can be entered as `hours tuesday` or `hours 2026-11-03`.
- An edit must supply at least one of its optional fields. Keep supplied fields in the order shown; omitted fields retain their values.

Input	Rule	Example
Commands and prefixes	Case sensitive; command words and field prefixes are lowercase	`task add`, `n/`, `due/`, `d/`
Fields	Use the stated order and each prefix at most once	`n/NAME due/YYYY-MM-DD HHmm d/MINUTES`
Names	Nonblank; may contain spaces and Unicode, but no `/`, tabs, newlines, or other control characters	`CS2113 draft`
Dates	Real dates in `YYYY-MM-DD` format; years `0001`–`9999`	`2026-10-06`
Task/commitment times	Four digits in 24-hour `HHmm` format, from `0000` to `2359`	`0930`, `1800`
Study hours	`HH:mm`, on half-hour boundaries; start before end within the same day	`09:00`, `22:30`
Deadlines	Date and time separated by a space; any minute is allowed	`2026-10-06 2359`
Task durations and remaining estimates	Positive whole minutes; no sign or leading zeroes	`1`, `45`, `180`
Weekly task template durations	Positive whole minutes divisible by 30	`30`, `90`, `180`
Commitment durations	Positive whole minutes divisible by 30; no sign or leading zeroes	`30`, `90`, `180`
Commitment starts	Must be on the hour or half hour	`1000`, `1030`
Commitment weekdays (`DAY`)	Exactly `MON`, `TUE`, `WED`, `THU`, `FRI`, `SAT`, or `SUN`	`MON`
Study-hours weekdays (`DAY`)	Full lowercase weekday names: `monday` through `sunday`	`tuesday`

Input	Rule	Example
Task IDs	`T` followed by a positive integer, without leading zeroes	`T1`, `T12`
Recurring task template IDs	`R` followed by a positive integer, without leading zeroes; separate from task occurrence IDs	`R1`, `R12`
Commitment IDs	`C` followed by a positive integer, without leading zeroes	`C1`, `C12`

Use one or more ordinary spaces between words and fields; embedded tabs are rejected. Outer whitespace is trimmed, and blank lines are ignored. Names are trimmed at both ends while retaining internal spaces. Quotation marks are stored as part of the name, so enter `n/CS2113 draft` without surrounding quotes. Duplicate names are allowed.

IDs identify records, not list positions. Bare numbers such as `1`, zero-padded IDs such as `T01`, and IDs with the wrong prefix are rejected. Tasks, commitments and recurring task templates have separate sequences starting at `T1`, `C1` and `R1`. Deleted IDs are never reused, including after a restart; a failed add does not consume an ID.

Whole-number inputs use digits `0`–`9`, without signs or leading zeroes, and must not exceed `2147483647`. Zero is accepted only where stated, such as `done/0` or `percent/0` in progress commands. An exhausted ID sequence rejects further additions.

Missing required fields and repeated, unknown or reordered prefixes are rejected. Do not append arguments beyond the format shown. For example, `help`, `plan` and `exit` accept no arguments. `task list` and `task list completed` are two supported forms; other trailing words are rejected. For commitments, `on/` cannot be combined with `day/`, `from/` or `until/`.

Impossible dates such as `2026-02-30` are rejected rather than adjusted. A new task deadline must be in the future and within the allowed date range. Commitment starts and durations must also satisfy the commitment rules. Correct rejected input at the next prompt; see command troubleshooting.

Examples below are independent unless a sequence is explicitly described. Use existing IDs and suitable future task deadlines in your own session. Output times contain a colon (`10:30`), while task/commitment command times use four digits (`1030`).

Tasks

> **Candidate-release note:** these Task sections describe the upcoming-release source and its task/parser tests, not every feature in the published v1.0 JAR. Use the matching candidate build when trying progress, completion and reopening.

A task records exact whole minutes of remaining work. A positive estimate is unfinished; zero means **completed but retained**, not deleted. Neither planning nor elapsed time subtracts work automatically. Use `task progress` to record work, `task remaining` to replace an estimate, `task complete` to finish it, and `task reopen` to restore unfinished work. `task list` includes retained completed records; `task list completed` shows only completed ones.

Task IDs such as `T1` remain stable. Editing, progress, completion and reopening do not replace them. Successful changes save automatically and clear the saved plan; run `plan` again. Recognized no-ops and rejected commands preserve it. Loaded undated tasks display `No deadline`, but console addition requires `due/`; there are no console commands for clearing deadlines or changing priority. For repeated coursework, see weekly task templates.

Recording duration (tasks and commitments)

Use `d/MINUTES` when adding a task or commitment; there is no separate duration command or running timer. Tasks accept positive whole minutes, such as `d/1` or `d/45`. Commitments require positive multiples of 30, so commitment `d/45` is rejected. Both additions reject `d/0`.

For a task, duration is the **estimated remaining work**, which may be split across sessions. For a commitment, it is the length of each occurrence from its start time; the interval must not cross midnight. Both values survive restart.

Planning reserves half-hour slots without changing those stored values. A 45-minute task needs 60 minutes of schedule space but remains 45 minutes of work. If only 30 minutes fits, 15 minutes remains unallocated and needs another 30-minute slot. Viewing a plan or letting time pass does not record progress.

Use progress to subtract work actually completed. Its percentage form describes the reported session, not the whole task. Completion retains a task with zero remaining minutes; use the reopening command to give it a new estimate.

To correct an active task's estimate without recording completed work:

```
scheduleflow> task remaining T1 remaining/90
Remaining time for T1: CS2113 draft | 90 min
```

Use an existing active task ID and a positive whole-minute estimate; `remaining/45` is accepted. A changed estimate saves automatically and clears the plan; run `plan` again. Repeating the same value reports the stored estimate without another save or plan invalidation. Unknown IDs, invalid values, attempts to revise a completed task and failed saves preserve the prior data and plan. For example, if a save fails while changing 45 to 90 minutes, the stored estimate remains 45 until a retry succeeds.

Recording progress

```
task progress TNUMBER done/MINUTES
task progress TNUMBER session/MINUTES percent/PERCENT
```

These examples are a sequence for an **active T1 named CS2113 draft with 90 minutes remaining**. Substitute an existing task ID from `task list`:

```
scheduleflow> task progress T1 done/30
Recorded progress for T1: CS2113 draft | Credited: 30 min | Remaining: 60 min
scheduleflow> task progress T1 session/61 percent/25
Recorded progress for T1: CS2113 draft | Credited: 15 min | Remaining: 45 min
```

`done/` reports the actual work completed, from zero through the remaining estimate. The session form requires positive whole minutes and an integer percentage from **0 to 100 inclusive**. It describes that session, not a percentage of the whole task. Credit is rounded down: 25% of 61 minutes credits 15, not 16. Both forms display the actual credited minutes and new remaining estimate. Use decimal digits without signs or leading zeroes; minute values must fit in a signed 32-bit integer.

Credit above the remaining estimate is rejected, not silently limited. After the sequence above leaves 45 minutes, `task progress T1 done/46` reports an error and leaves 45. `done/0`, `percent/0`, or a session percentage rounding to zero reports zero credit **without another save or plan invalidation** on an active task. Positive credit saves before success, clears the old plan and survives restart. Run `plan` again to schedule the revised estimate.

Crediting all remaining work retains the task at `0 min`. Further progress, including `done/0`, is rejected until you reopen it. Each successful repeated submission records additional work; there is no automatic duplicate detection. A failed save applies nothing, so retrying that failed request is safe. Unknown IDs, invalid values and rejected saves preserve the prior records and plan. Progress on a generated task changes only that occurrence, not its template or original release time. It remains available after the template is deleted.

Adding a task

Command: `task add`

```
task add n/NAME due/YYYY-MM-DD HHmm d/MINUTES
```

Supply a nonblank name, a future local deadline and a positive whole-minute estimate. `d/1` and `d/45` are valid for tasks; `d/0`, negative values and leading-zero tokens such as `d/045` are not. A deadline may be any minute, including `2359`; it need not lie on a half-hour boundary.

The deadline must be **strictly after the command's current time**, and no later than midnight at the start of the same calendar date next year. For example, on 26 October 2026 the inclusive upper limit is **26 October 2027 at 00:00**. See planning dates for the calendar-year and leap-day rules.

```
scheduleflow> task add n/CS2113 draft due/2026-11-20 1800 d/45
Added T1: CS2113 draft
Due: 2026-11-20 18:00 | Duration: 45 min
```

Replace the example deadline with a future date in your allowed range. In a fresh store the first ID is T1; otherwise use the ID the app reports. IDs are not list positions and deleted IDs are never reused. Duplicate names are allowed. You cannot omit `due/` or use `due/NONE` at the console. For example:

```
scheduleflow> task add n/CS2113 draft d/180
Error: Invalid command format or missing fields
Expected: task add n/NAME due/YYYY-MM-DD HHmm d/MINUTES
```

A successful addition saves before printing its confirmation and clears any current plan. Run `plan` again to include it. Rejected additions consume no ID.

Rejected task additions leave existing tasks, commitments, ID counters and the current plan unchanged. Correct the input and retry; the failed attempt does not consume an ID. Optional Task diagnostics record validation and candidate-processing stages in a separate log file, without task names, deadlines or durations. Normal launches keep the same console output.

Listing tasks

```
task list
```

Lists **all stored tasks**, including overdue and completed records, in numeric ID order. The existing heading `Actionable Tasks` does not filter out completed records. For example, these stored tasks produce:

```
scheduleflow> task list
=== Actionable Tasks ===
ID | Name | Duration | Deadline
T1 | CS2113 draft | 45 min | 2026-11-02 18:00
T2 | Reading | 0 min | 2026-11-02 18:00
Total: 2 task(s).
```

An empty store displays `No tasks found.` Use `task list completed` for only the retained completed records. Template R IDs appear in `task recur list`; generated occurrences appear here with ordinary T IDs.

Duration is the saved remaining-work estimate, not the unallocated remainder of the latest plan. Listing neither credits work nor changes the saved plan.

Editing a task

```
task edit TNUMBER [n/NAME] [due/YYYY-MM-DD HHmm]
```

Supply a name, a deadline, or both; an empty edit is rejected. When both are present, `n/` must precede `due/`. Omitted fields retain their values. The example assumes T1 exists with 180 minutes remaining:

```
scheduleflow> task edit T1 n/CS2113 final draft due/2026-11-20 1800
Edited T1: CS2113 final draft
Due: 2026-11-20 18:00 | Duration: 180 min
```

Replace the sample date with a suitable future date. Only an **actually changed** deadline must be future and within the rolling calendar-year limit. You may rename an overdue task, or explicitly keep its unchanged old deadline. To rename only, enter `task edit T1 n/CS2113 final draft`.

Edits preserve ID, stored position, remaining work, completion status, priority and any recurring-task provenance. Editing one generated task does not edit its template or original release time. A changed edit saves and clears the plan; run `plan` again. A name that normalizes to the existing value, with no other effective change, reports the same details without saving or clearing the plan. Unknown IDs, invalid replacements and failed saves leave the whole edit unapplied. To revise the estimate use `task remaining T1 remaining/90`, not `d/` in an edit. Removing a deadline with `due/NONE` is not supported by the console.

Weekly recurring tasks

```
task recur add n/NAME due/YYYY-MM-DD HHmm d/MINUTES every/WEEKLY [until/YYYY-MM-DD]
task recur list
task recur delete RNUMBER
```

A template describes unfinished work repeated each week, not a fixed busy commitment. Its first deadline fixes the weekday and time of all occurrences. The first deadline must be in the future. Estimates remain positive multiples of 30 minutes. Optional `until/` is an inclusive last due date, on or after the first due date; omit it for no end date. Only `every/WEEKLY` is supported.

```
scheduleflow> task recur add n/Weekly quiz due/2026-11-02 1800 d/60 every/WEEKLY until/2026-11-30
Added R1: Weekly quiz
Weekly from: 2026-11-02 18:00 | Duration: 60 min | Until: 2026-11-30
```

Use future dates in your own session. `task recur list` displays the stable `R` IDs and definitions; an empty list prints `No recurring tasks found.` Each materialized occurrence gets its own `T` ID and appears in `task list`. Adding a template materializes future deadlines through the rolling planning horizon. Each `plan` command extends generation as that horizon moves forward. Templates may start beyond the current horizon; their first tasks appear once eligible. A task is never scheduled before one week before its original due date, even if you later edit that occurrence's deadline. A generated task can appear in `task list` before its release time; planning simply waits until the release time to schedule it.

Generation skips ungenerated deadlines already passed; it does not backfill missed weeks. Saved cursors prevent restart, replanning or deleting an occurrence from recreating it. Templates, provenance and all counters survive saved restarts and unrelated task/commitment/study-hours changes. Template IDs are never reused. Failed validation, planning or saving leaves live state unchanged.

`task recur delete R1` stops future generation but **retains every already generated task**, including future occurrences. Remove unwanted occurrences separately with `task delete TNUMBER`. Editing a generated task affects only that task, not its template or original release time. Template editing and daily/monthly recurrence are not supported.

Generated tasks use the same exact remaining-work and lifecycle model as ordinary tasks. Their estimates can be revised with `task remaining`, and completed work can be recorded with `task progress`; both preserve their provenance. Explicit completion and reopening are available through `task complete` and `task reopen`.

Completing and viewing completed tasks

Use `task complete TNUMBER` when no work remains, even if the stored estimate is still positive. It sets remaining work to zero **without deleting the task**. The changed task saves automatically and clears the plan; run `plan` again to exclude it from sessions and unallocated-work reports. It retains its ID, name, deadline, priority and any recurring-task provenance.

The following assumes T1 named CS2113 draft has deadline 2026-11-20 18:00:

```
scheduleflow> task complete T1
Completed T1: CS2113 draft
scheduleflow> task list completed
=== Completed Tasks ===
ID | Name | Duration | Deadline
T1 | CS2113 draft | 0 min | 2026-11-20 18:00
Total: 1 task(s).
```

`task list completed` shows only zero-work records in numeric ID order, or `No completed tasks found.` when empty. `task list` still includes them. Listing makes no data/plan changes. Repeating `task complete T1` on an already completed record is a no-op, preserving the plan without another save. Unknown IDs and failed saves preserve the previous record and plan. Completion survives restart; use reopen, not a new task, to correct it.

Reopening a completed task

```
task reopen TNUMBER remaining/MINUTES
```

If you completed a task by mistake, reopen the **existing completed record** with a new estimate. Supply positive whole minutes, including `1` or `45`, without signs or leading zeroes. Completion retained zero, not an earlier estimate to restore, so the amount must be explicit. The example assumes a completed T1 named CS2113 draft:

```
scheduleflow> task reopen T1 remaining/45
Reopened T1: CS2113 draft | Remaining: 45 min
```

Completion may have come from `task complete` or progress credit reaching zero. Reopening preserves the ID, name, deadline, priority and any recurrence provenance; it does not edit a template or generate another occurrence. You can also reopen a retained generated task after deleting its template. Loaded completed tasks with overdue or absent deadlines may also reopen unchanged in those respects. An overdue task remains overdue until you change its deadline.

Saving succeeds before the task becomes active and clears the current plan. Run `plan` again to include the revised work. The task no longer appears in `task list completed`, and its remaining estimate survives restarting. Unknown IDs, active tasks, invalid estimates and failed saves preserve the existing records and plan. To change an active task's estimate instead, use `task remaining TNUMBER remaining/MINUTES`.

Deleting a task

```
task delete TNUMBER
```

Deletes the task permanently. Use this command to remove completed or unwanted work; there is no undo. Prefer `task complete` if you want to keep a completed record, or `task edit` to correct its details without replacing the ID. Use the exact T ID shown by `task list`, not a row number or a recurring template's R ID.

```
scheduleflow> task delete T1
Deleted task T1: CS2113 draft
```

A successful deletion saves and clears the plan; run `plan` again. Deleted IDs are not reused, even when the task list becomes empty or after restart. An unknown ID, malformed command or failed save leaves the data and plan unchanged. Deleting a generated occurrence does not delete its template, and its saved generation history prevents that same occurrence from being recreated. To stop future generation, see template deletion.

Commitments

A recurring commitment reserves the same period every week. It can continue until deleted, or use inclusive start and end dates for a semester or other limited period. Add separate entries for commitments on different weekdays. A one-time commitment, such as a project meeting, reserves a period on a single calendar date.

Commitment addition, editing, skipping, deletion and listing run directly at the console. Each successful change saves automatically and clears the current plan.

Adding a commitment

```
commitment add n/NAME day/DAY start/HHmm d/MINUTES [from/YYYY-MM-DD until/YYYY-MM-DD]
```

```
scheduleflow> commitment add n/CS2113 lecture day/MON start/1000 d/120
Added C1: CS2113 lecture | Monday 10:00 - 12:00 (120 min)
```

This blocks 10:00–12:00 every Monday.

To limit the series to a semester, provide both dates:

```
scheduleflow> commitment add n/CS2113 lecture day/MON start/1000 d/120 from/2026-08-10 until/2026-11-16
Added C1: CS2113 lecture | Monday 10:00 - 12:00 (120 min) | 2026-08-10 to 2026-11-16
```

Both endpoints are inclusive. The range must contain at least one occurrence of the chosen weekday. Supply both `from/` and `until/`, in that order, or omit both for an unbounded weekly series. Study time is blocked only on matching weekdays inside the range.

- The start must be on `:00` or `:30`, and the duration must be a positive multiple of 30 minutes.
- A commitment must end by midnight on its selected day. Ending exactly at midnight is allowed and displayed as `24:00`; entering `2400` as a start time is invalid.
- Commitments may occur outside study hours. A full-day commitment starting at `0000` with duration `1440` is valid.
- Commitments on the same weekday must not overlap when their actual occurrences share a date, even outside study hours. Separate semester ranges may reuse the same weekly time. An overlap error identifies every conflicting commitment by ID and name, in start-time order.
- Adjacent commitments are allowed: Monday 10:00–12:00 and Monday 12:00–13:00 do not overlap. The same time on different weekdays is also allowed.

A successful addition saves the commitment and clears the current plan.

If an addition overlaps multiple entries, the error lists them together, for example: `Error: Commitment overlaps C10: Lecture; C2: Seminar`. Adjacent entries and commitments on other weekdays are not included in that list. Correct the proposed interval and retry; rejected additions do not consume an ID.

Rejected additions leave existing commitments and IDs unchanged. Optional commitment diagnostics record processing stages separately without commitment names or schedules. Normal launches keep the same console output.

Adding a one-time commitment

```
commitment add n/NAME on/YYYY-MM-DD start/HHmm d/MINUTES
```

Use `on/` instead of `day/` for an event that happens once:

```
scheduleflow> commitment add n/Project meeting on/2026-10-30 start/1400 d/60
Added C2: Project meeting | One-time on 2026-10-30 (Friday) 14:00 - 15:00 (60 min)
```

This blocks 14:00–15:00 on 30 October 2026 only; other Fridays stay available. The weekday is taken from the date, so do not also type `day/`, `from/` or `until/`.

- The date must be a real calendar date from `0001-01-01` to `9999-12-31`.
- The time and duration rules are the same as for recurring commitments.
- A one-time commitment must not overlap a recurring commitment that occurs on that date, or another one-time commitment on the same date. Adjacent times are allowed. A recurring occurrence that you skipped does not cause a conflict.
- Adding a recurring commitment is likewise rejected if it would overlap an existing one-time commitment.

One-time commitments receive ordinary IDs (`C1`, `C2`, ...) from the same sequence as recurring commitments, and are listed, edited and deleted with the same commands. Rejected additions do not consume an ID. A successful addition saves automatically and clears the current plan. A recurring commitment entered with the same `from/` and `until/` date has exactly one occurrence, so it is treated and displayed as a one-time commitment.

Editing a commitment

```
commitment edit CNUMBER [n/NAME] [day/DAY|on/YYYY-MM-DD] [start/HHmm] [d/MINUTES] [from/YYYY-MM-DD
until/YYYY-MM-DD]
```

Supply at least one field, in the order shown; do not type the square brackets. Omitted fields retain their stored values. The existing ID is kept, and editing does not consume a new ID or change other commitments. This edits the whole weekly series, not just one occurrence. Supplying `from/` and `until/` together replaces its date range. Omitting them preserves the existing range. Removing an existing range is not supported yet.

```
scheduleflow> commitment edit C1 start/1100 d/60
Edited C1: CS2113 lecture | Monday 11:00 - 12:00 (60 min)
```

You can rename with `commitment edit C1 n/New lecture` or move the series with `commitment edit C1 day/TUE`. The full resulting interval must satisfy the same rules as addition. Conflict checks exclude the edited series itself, but still check every other series. Blank names, unknown IDs and empty edits are rejected. When changing a late start and duration, supply both together so the complete replacement ends by midnight.

For a one-time commitment, `n/`, `start/` and `d/` work as above. Move it to another date with `on/`, which also sets the weekday:

```
scheduleflow> commitment edit C2 on/2026-11-06
Edited C2: Project meeting | One-time on 2026-11-06 (Friday) 14:00 - 15:00 (60 min)
```

`day/` alone is rejected for a one-time commitment because its weekday comes from its date. `on/` cannot be combined with `day/`, `from/` or `until/`. Using `on/` on a recurring commitment turns it into a one-time commitment on that date, and its skipped dates are discarded because it no longer repeats. Supplying `from/` and `until/` for a one-time commitment turns it into a bounded weekly series.

Successful edits save automatically and clear the plan; run `plan` again. This also applies when you explicitly supply an existing value. Invalid edits and failed saves leave the saved data, IDs, current records and plan unchanged. Correct the input or save problem and retry. Edited values survive restarting the application. Editing a commitment also preserves configured study hours and date overrides.

Skipping one occurrence

```
commitment skip CNUMBER on/YYYY-MM-DD
```

Use this when one class or meeting is cancelled but the rest of its recurring series remains unchanged:

```
scheduleflow> commitment skip C1 on/2026-10-26
Skipped C1 on 2026-10-26: CS2113 lecture
```

The date must be an unskipped occurrence of that commitment: it must use the same weekday and, for a bounded series, fall within its inclusive range. An unknown ID, a date on another weekday, a date outside the range or an already skipped date is rejected without changing saved data or the current plan.

A successful skip saves automatically and clears the current plan. Run `plan` again; the released period is then available for study. Other commitments may use the same time on that skipped date, while conflict checks still apply on every remaining occurrence. Skips survive restarting the application and are shown by `commitment list`. Delete the series if every occurrence should be removed instead. One-time commitments cannot be skipped; delete them instead.

Listing commitments

```
commitment list
```

Lists all stored commitments, including those outside study hours or with past dates. Listing changes neither saved data nor the current plan. Use the displayed `C` IDs for edits, skips and deletions; they are not renumbered to match the list order.

Weekly series appear first, grouped Monday through Sunday, then ordered by start time and numeric ID. Bounded series show their inclusive date range after the name. Skipped dates appear in chronological order after `| skipped:`. One-time commitments follow in a separate section, ordered by date, start time and numeric ID. Empty sections and weekday groups are omitted.

For this example, start with an empty store so the IDs begin at `C1`, and run these setup commands in order:

```
commitment add n/CS2113 lecture day/MON start/1000 d/120 from/2026-11-02 until/2026-11-30
commitment add n/EE2026 lecture day/MON start/1200 d/60
commitment add n/Weekly review day/FRI start/2330 d/30
commitment add n/Project meeting on/2026-11-06 start/1400 d/60
commitment skip C1 on/2026-11-16
commitment skip C1 on/2026-11-09
```

Then list the saved entries:

```
scheduleflow> commitment list
=== Recurring Weekly Commitments ===
[Monday]
C1. 10:00 - 12:00 | CS2113 lecture | 2026-11-02 to 2026-11-30 | skipped: 2026-11-09, 2026-11-16
C2. 12:00 - 13:00 | EE2026 lecture

[Friday]
C3. 23:30 - 24:00 | Weekly review

Total: 3 recurring commitments across the week.

=== One-time Commitments ===
C4. 2026-11-06 (Friday) 14:00 - 15:00 | Project meeting
Total: 1 one-time commitments.
```

`24:00` means midnight at the end of the commitment's day. A weekly entry with identical `from/` and `until/` dates has one occurrence and appears in the one-time section. Skipped occurrences do not remove the weekly definition or reduce its section's commitment count.

An empty list displays:

```
No commitments found.
```

Deleting a commitment

```
commitment delete CNUMBER
```

Removes the entire weekly series, not just one occurrence, or removes a one-time commitment.

Use the original `CNUMBER` shown by `commitment list`, not its position in the display. An unknown ID is rejected without changing commitments or ID counters. Deleting the final commitment leaves an empty list but does not reset its ID sequence; a later addition receives a new ID.

```
scheduleflow> commitment delete C1
Deleted commitment C1: CS2113 lecture
```

A successful deletion saves the change and clears the current plan. Run `plan` again to make the released time available for study sessions.

Study hours

Set a recurring window independently for each weekday:

```
hours tuesday /start 09:00 /end 22:00
hours wednesday /start 10:00 /end 18:00
```

Override a single calendar date without changing its recurring weekday setting:

```
hours 2026-10-27 /start 14:00 /end 19:00
```

The date override takes priority over the weekday setting, which takes priority over the default 08:00–22:00 window. Thus October 27 uses 14:00–19:00, other Tuesdays use 09:00–22:00, Wednesdays use 10:00–18:00, and other days keep the default.

View the effective window or remove only the selected setting:

```
hours tuesday
hours 2026-10-27
hours reset 2026-10-27
hours reset tuesday
```

Resetting a date restores its weekday hours (or the default). Resetting a weekday restores its default but preserves date overrides. Resetting an unset target is allowed. Viewing hours does not require or change a plan.

Reset confirmations explicitly show the restored hours. For example, `hours reset tuesday` reports `Using default hours: 08:00 - 22:00.` A date reset reports `Using recurring weekday hours` instead when that weekday has a configured setting, followed by its actual start and end times.

Times must use `HH:mm` and fall on `:00` or `:30`, from `00:00` through `23:30`. Start must be strictly before end on the same day. Overnight, zero-length and `24:00` windows are unsupported. Each day has one continuous study window; use commitments to block breaks within it. Dates may be any valid date in years 0001–9999; configuring hours does not extend the planning horizon.

Successful sets and resets save automatically and clear the current plan. Run `plan` again to apply the change. Invalid commands and failed saves preserve both settings and the current plan. Task and commitment changes preserve hours, and settings survive restarting the application. Existing saves without hours continue to use the default. A save containing custom hours requires this version of ScheduleFlow; older versions cannot read the new format.

Study plans

A study plan is a saved snapshot of your tasks, commitments, study hours and policy when you run `plan`. Use `schedule` to view its time slots by date. The plan does not change a task's saved remaining-work estimate. Every newly generated result is checked against the tasks and availability captured for that plan before it is returned for saving.

Generating a plan

```
plan
```

ScheduleFlow excludes completed tasks and allocates dated tasks by earliest deadline first, with stored priority (HIGH, NORMAL, LOW) breaking deadline ties, then numeric ID. All console-created tasks default to NORMAL, so `T2` still comes before `T10` at an equal deadline. Loaded undated tasks follow all dated tasks, by priority and ID, within the same horizon. Priority operations are not yet available at the console.

Each task fills the earliest available 30-minute slots that end at or before its deadline. Work may split around commitments or across days. Adjacent slots for the same task on the same day are displayed as one session. Sessions never overlap commitments or other sessions.

If a task cannot fit completely, `plan` still saves the sessions that fit and reports its remaining work. See Unallocated work for the reported reasons and how exact work relates to the half-hour slots.

When every task fits:

```
scheduleflow> plan
Study plan generated successfully! Type 'schedule today' to view your sessions.
```

When there are no tasks:

```
scheduleflow> plan
No tasks to schedule.
```

Even with no tasks, `plan` creates a current plan. You can then view commitments and free or unplanned periods using `schedule`.

Each `plan` command saves and replaces the previous plan without changing stored task durations. If saving fails, the previous plan remains available. This allocator follows deadline order; it does not guarantee the fewest sessions or the greatest number of fully completed tasks.

Marking a study period unavailable

Use a one-time commitment to block time on a particular date, including time that a previous plan assigned to a task:

```
commitment add n/Unavailable on/2026-11-04 start/1000 d/60
plan
schedule 2026-11-04
```

Replace the example date with your chosen date within the current planning range. This blocks 10:00-11:00 on that date only. You can choose any descriptive name, such as `Appointment`; `Unavailable` has no special meaning. The period appears as `[BUSY]` with its commitment ID. Replanning moves task work into other eligible slots and reports unallocated minutes if it cannot all fit before the deadlines. Stored task estimates do not change.

Adding the block saves automatically and clears the old plan; run `plan` again to apply it. Both the block and the newly generated plan survive restarting. You can release it with `commitment delete CNUMBER`, using the ID shown by `commitment list`, then run `plan` again. Successful deletion also saves and clears the current plan.

The existing one-time commitment rules apply: half-hour-aligned starts, positive durations divisible by 30, and an interval ending by midnight. A block may overlap planned task sessions, but it cannot overlap another commitment occurring on that date. Rejected additions preserve the existing plan and data. There is no separate `unavailable` command or command to select a session by ID.

Blocking multiple dates

Use either command to block several dates in one atomic request:

```
unavailable continuous n/Trip from/2026-10-26 1500 until/2026-10-28 1100
unavailable daily n/Appointment from/2026-10-26 until/2026-10-28 start/1500 end/1700
```

Continuous mode blocks every moment from the first date/time up to, but excluding, the last date/time. It splits the interval at midnight into one-time commitments. An exclusive endpoint at midnight does not create an empty period on that final date. For example:

```
scheduleflow> unavailable continuous n/Trip from/2026-11-02 1500 until/2026-11-04 1100
Blocked 3 unavailable periods.
C1 | 2026-11-02 | 15:00 - 24:00 | Trip
C2 | 2026-11-03 | 00:00 - 24:00 | Trip
C3 | 2026-11-04 | 00:00 - 11:00 | Trip
Saved unavailable periods. Run plan to regenerate.
```

Daily mode blocks only the stated same-day window on every date from `from/` through `until/`, both inclusive. Its start must be earlier than its end; use continuous mode for an overnight interval. Both modes require half-hour-aligned times and ordinary commitment names.

Each generated period receives its own `C` ID, so it can later be edited or deleted independently. ScheduleFlow validates the complete batch first. If any period conflicts, the date range is invalid, IDs are insufficient, or saving fails, it adds nothing, consumes no IDs and preserves the current plan. A successful batch is saved once, clears the previous plan and survives restart; run `plan` again before viewing the changed schedule.

The Developer Guide transaction description explains the all-or-nothing integration and verification boundaries.

Session limits and breaks

Set a maximum continuous-work duration and the consecutive break required before further study. Both values must be positive whole multiples of 30 minutes.

```
scheduleflow> policy set max/90 break/30
Study policy: Maximum continuous work: 90 min | Required break: 30 min
Saved study policy. Run plan to regenerate.
scheduleflow> policy show
Study policy: Maximum continuous work: 90 min | Required break: 30 min
scheduleflow> policy reset
Study policy: unrestricted (no session limit or required break).
Saved study policy. Run plan to regenerate.
```

Settings save automatically and survive restarting. Changing them clears the saved plan; run `plan` to generate a schedule using the new values. Repeating a set with the same values, or resetting an already-unrestricted policy, prints `Study policy unchanged.` and preserves the plan. Invalid input and failed saves preserve both settings and the current plan. `policy show` changes nothing.

The continuous-work limit counts study tasks and fixed commitments together. At the limit, further study waits for the configured consecutive break, shown as `HH:mm - HH:mm | [BREAK]`. Commitments stay fixed even when they exceed the limit; one configured break still suffices afterward. Natural idle time can satisfy the break. Study-session padding counts as occupied time. With no stored policy, including when opening an older file, planning remains unrestricted.

Planning hours and date range

Study windows default to **08:00–22:00 every local calendar day**, including weekends. The planner uses each date's effective study hours.

For the day on which you generate a plan, scheduling starts at the later of the configured start and the current time rounded up to the next half hour. At or after the configured end, no study slots remain that day. Future days start at their configured start. The following examples use default hours.

Time when `plan` runs	Earliest eligible start that day
07:00	08:00
10:00:00 exactly	10:00
10:00:01	10:30
10:07	10:30
21:45	No slots remain; cutoff is 22:00

The planning limit is **midnight at the start of the same calendar date next year**, recalculated for each `plan` command. The plan includes the generation date through the day before that limit. Task additions use the same limit, calculated when the task is added.

For example, on 5 October 2026:

- A new task's deadline must be later than the current time and at or before **5 October 2027 at 00:00**.
- A new plan covers **5 October 2026 through 4 October 2027**, inclusive.

- With default hours, the latest possible study session ends at **22:00 on 4 October 2027**.

This is a calendar year, not a fixed 365 days. For a 29 February start date, the following year's boundary adjusts to 28 February. There is no first-startup limit and no need to delete saved data to move the range forward.

Dates and times use your computer's local wall time. Time zone offsets are not stored; changing the computer's time zone reinterprets saved times as local times. Durations are wall-clock minutes, including across daylight-saving changes.

Unallocated work

If work does not fit, ScheduleFlow keeps a **partial plan** and reports the remaining minutes for each affected task. This is a valid planning result. Even a completely blocked task remains in the plan's unallocated-work report.

For example, if a 90-minute task is due at 08:30 and planning runs before 08:00 that day with no commitments:

```
scheduleflow> plan
Study plan generated with unallocated work.
Scheduled: 30 min | Unallocated: 60 min
T1: Draft | Unallocated: 60 min | Reason: insufficient time before deadline
```

The task still has a stored duration of 90 minutes. Only 30 minutes were allocated in this plan.

Reported reason	Meaning
`deadline passed`	The task's deadline is at or before the time the plan was generated. Its full duration remains unallocated.
`outside planning horizon`	The deadline is later than this plan's date limit, for example after restoring data or changing the clock. Its full duration remains unallocated.
`insufficient time before deadline`	Too few eligible slots remain before the deadline. Any slots that do fit are retained.

The totals cover the whole plan. Unallocated tasks appear in deadline order, with priority then numeric ID breaking ties; undated tasks follow dated tasks. Slots must fit in full: a deadline of 08:59 permits an 08:00–08:30 session but not an 08:30–09:00 session.

Summary scheduled/unallocated totals count exact work, excluding padded capacity. Schedule rows still show reserved interval durations. Undated remainder rows say `No deadline` and `insufficient available study time`, never `deadline passed` or `outside planning horizon`. They follow dated rows; priority breaks equal deadline ties before numeric ID. Completed overdue tasks contribute no remainder.

For example, a task with 45 minutes remaining needs two half-hour slots. The plan credits 45 minutes of work while the schedule reserves 60 minutes; the remaining 15 minutes are padding, not extra task work. Planning leaves the saved estimate unchanged; use `task progress` or `task remaining` to update it.

Viewing the schedule

If no plan exists, generate one with `plan`. Then use either command to view a date:

```
schedule today
schedule YYYY-MM-DD
```

`today` means the computer's current local date when the command runs. A dated command selects that calendar date explicitly. The selected date must be within the saved plan's date range. If a successful change has cleared the plan, run `plan` again before viewing it.

The schedule displays the latest saved plan's full configured study window in chronological order, along with its generation time. Viewing a date does not regenerate the plan or record progress. Commitments are clipped to that date's study window: with default hours, a 07:30–09:00 commitment appears as 08:00–09:00. Commitments entirely outside study hours remain visible in `commitment list` but do not appear in the schedule.

Schedule labels

Label	Meaning
`[TASK]`	A planned study session, showing its task ID, name and reserved interval duration.
`[BUSY]`	A recurring or one-time commitment, showing its commitment ID and name.
`[FREE]`	Eligible study time left after assigning tasks, blocking commitments and reserving required breaks.
`[UNPLANNED]`	Time before the rounded planning cutoff on the day the plan was generated. It is unavailable for new study sessions in this plan.
`[BREAK]`	A reserved rest interval under the configured work/break policy; configure it with `policy set`, then run `plan`.

A commitment remains `[BUSY]` even when it occurred before the planning cutoff. Adjacent entries are combined only when their label, ID, and name match, so separate commitments remain separate rows.

Task rows show reserved time in 30-minute slots. For example, 45 minutes of remaining work can occupy a 60-minute `[TASK]` interval. The extra 15 minutes are padding; generating or viewing the plan does not reduce the saved work estimate. See exact work and unallocated totals.

Worked schedule example

This reference transcript assumes an empty store, default **08:00–22:00** study hours, no work/break policy, and a local time of **Monday, 2 November 2026 at 07:00**. Run these commands in order:

```
task add n/CS2113 draft due/2026-11-03 1800 d/180
commitment add n/CS2113 lecture day/MON start/1000 d/120
commitment add n/EE2026 lecture day/MON start/1200 d/60
plan
schedule today
```

The schedule is:

```
=== Schedule for Monday (2026-11-02) ===
Generated at: 2026-11-02 07:00
08:00 - 10:00 | [TASK] T1: CS2113 draft (120 min)
10:00 - 12:00 | [BUSY] C1: CS2113 lecture
12:00 - 13:00 | [BUSY] C2: EE2026 lecture
13:00 - 14:00 | [TASK] T1: CS2113 draft (60 min)
14:00 - 22:00 | [FREE]
```

The task uses the two free morning hours first, then resumes after the commitments. All 180 minutes fit, so there is no unallocated-work footer.

When trying this at another time, use a future deadline within the allowed range. The displayed date, generation time and available slots depend on when you run `plan`; the exact rows above require the stated setup.

Planning partway through the day

This is an illustrative fixed-time example, not a clock-setting command. Suppose you generate a plan at **10:07 on 5 October 2026**, using default 08:00–22:00 hours, no policy or commitments, and one 30-minute task `Draft` due at 11:00 that day. The next usable half-hour boundary is 10:30:

```
=== Schedule for Monday (2026-10-05) ===
Generated at: 2026-10-05 10:07
08:00 - 10:30 | [UNPLANNED]
10:30 - 11:00 | [TASK] T1: Draft (30 min)
11:00 - 22:00 | [FREE]
UNPLANNED periods were before the planning cutoff.
```

If `Draft` instead needs 90 minutes with that same deadline, only 30 minutes fits. The schedule also includes this footer for the **whole plan**, even when viewing another date:

```
Unallocated work for the whole plan:
T1: Draft | Unallocated: 60 min | Reason: insufficient time before deadline
```

The saved plan stays frozen. Restarting and entering `schedule 2026-10-05` shows the same generation time and rows; viewing it later does not move its sessions or regenerate recurring tasks. Use `plan` when you want a fresh allocation based on the current time.

Choose a date within the saved plan's range. An out-of-range request reports the allowed dates. If no plan exists, the response is `No current plan. Run plan to generate one.` Successful source changes clear the saved plan; rejected commands and recognized no-ops preserve it. See why schedules disappear for details.

Help and exit

Use these commands at the `scheduleflow>` prompt. Both preserve saved data and the current plan.

Viewing help

```
help
```

Displays `=== ScheduleFlow Command Help ===`, followed by every supported command format and a short description, then returns to the prompt. Enter `help` without arguments. See the linked command summary for each command's detailed instructions.

Exiting the program

```
scheduleflow> exit extra
Error: This command accepts no arguments
Expected: exit
scheduleflow> exit
Goodbye for now!
```

`exit` closes the application without another save because successful changes have already been saved. As shown above, extra arguments are rejected and leave the application running so you can correct the command.

End of input (EOF) also prints `Goodbye for now!` and exits normally without another save. In a Windows terminal, use Ctrl+Z then Enter; in a macOS/Linux terminal, use Ctrl+D on an empty input line.

If the terminal or redirected output stops accepting text, the application stops with failure status 1 and accepts no further commands. A command may have saved successfully before its confirmation failed to appear. Restart with working output and inspect the saved tasks, commitments or schedule before repeating that command.

Saving and restoring data

ScheduleFlow loads saved records and any frozen plan **before its first prompt**. It saves successful changes automatically; there is no `save` or `load` command. Tasks and their exact remaining work/completion, commitments and skipped dates, study hours, policy, recurring templates, occurrence history and independent T/C/R ID counters survive restart. Deleting records does not reset those counters.

The data is a UTF-8 **text file** at `data/scheduleflow.txt`, relative to the folder from which you launch the app. Use the same launch folder to reopen the same data. A definitely absent file starts an empty session without creating a folder; the first successful change or `plan` creates the saved data. Reading lists or simply exiting an empty session does not create it.

Generating a plan saves its complete source together with the exact generation time, task sessions, unallocated work and reserved breaks. Restart restores that result without reallocating work or advancing recurrence. A successful change to tasks, templates, commitments, hours or policy invalidates the saved plan. Recognized no-ops preserve it. Run `plan` again after a change.

Compatibility. The candidate reads snapshot formats 1–5 and 7, frozen-plan formats 6 and 8, and the earlier version 5 plan layout. Loading is read-only and does not migrate the file. A later successful save chooses the format needed for the current records: snapshot versions 2–5 or 7, or plan version 6/8. These numbers are file formats, not application release numbers. Older JARs may reject newer data; keep a backup before changing versions. The DG compatibility table documents the layouts.

If saving fails, the command reports an error. Its candidate records, counters and plan are not published; the previous live state and saved data remain in use. Resolve the file/folder problem and retry. Use one running instance per data file and a filesystem supporting atomic replacement. This does not guarantee recovery after a power failure.

The app attempts to remove a temporary save file after a failed write or replacement. If that cleanup also fails, a `.scheduleflow-*.tmp` file may remain beside `scheduleflow.txt`. It is not an automatic backup or a file the app loads. Close the app and preserve the main file before investigating; do not replace it with an incomplete temporary file.

If loading fails, the app reports an error and stops before the first prompt. It preserves the invalid/unreadable file rather than silently starting empty or accepting part of it. Valid overdue tasks still load; a new plan reports their remaining work as `deadline passed`.

A saved plan must also fit its saved availability, deadlines and remaining work. An impossible plan is rejected as a whole; the app preserves the file instead of silently dropping its schedule. Restore a known-good backup before restarting. These checks use the saved planning time, so an old but valid frozen plan can still be viewed without moving its sessions.

Backup or transfer: close the app, then copy `data/scheduleflow.txt` to a safe location. To transfer, put a copy in the destination launch folder's `data` subfolder, backing up any existing destination file first, and launch a compatible JAR there using Java 25. Confirm the records and saved schedule before discarding the original backup. The app does not maintain an automatic backup history.

Troubleshooting and FAQ

Ordinary command errors return you to the `scheduleflow>` prompt so you can correct the input and retry. A startup data-load error stops the application; follow the data recovery guidance before restarting.

What if my work does not fit before its deadline?

Run `plan` and review the unallocated minutes and reasons. The app retains every session that fits. Revise an estimate with `task remaining`, change a deadline with `task edit`, or remove a commitment if that time has become available. You can also review the configured study window with `hours DAY` or `hours YYYY-MM-DD`, or check the continuous-work policy with `policy show`. Adjust those settings when appropriate. Run `plan` again after any change to see an updated allocation.

How do I mark work as complete or correct an entry?

Use `task complete TNUMBER` to **retain** finished work at zero minutes, or `task progress` to subtract reported work; crediting the full remainder also retains completion. Correct mistaken completion with `task reopen TNUMBER remaining/MINUTES`, supplying a new positive estimate. Use `task delete TNUMBER` only for permanent removal, not as a prerequisite to completion. There is no undo for deletion.

Correct a task's name/deadline with `task edit TNUMBER` and `n/` and/or `due/`. Replace an active task's estimate with `task remaining TNUMBER remaining/MINUTES`; this may increase it and is not progress credit. Those corrections preserve the T ID. Commitment corrections use the separate `commitment edit CNUMBER` command. See the respective command sections for required fields and no-op rules.

Why did my schedule disappear?

A successful change to task details or remaining work, completion/reopening, templates, commitments (including skips), study hours or policy clears the current and saved plan. Those changes can make its sessions stale; use `plan` to generate another schedule.

Restarting alone preserves a valid frozen plan. Rejected commands and recognized no-ops also preserve it: for example, zero credited progress, repeating the same remaining estimate or setting an equal policy. If you launched from a different folder, check the data location before assuming your original schedule was lost.

Can I change the study hours or slot size?

Use `hours` to configure weekday hours and date overrides. The slot size remains fixed at 30 minutes; there is no command to change it. A final slot can reserve 30 minutes while crediting fewer minutes of exact task work.

Can I schedule more than a year ahead?

An ordinary task's new or changed deadline must be within the rolling calendar-year limit when that command runs. Wait until it enters the range, rather than deleting your saved data. The boundary is midnight at the start of the same local date next year, with February 29 adjusted to February 28. Each `plan` recalculates its own range; see planning dates. Weekly templates can begin beyond that range and materialize tasks when eligible; their separate rules are in weekly recurring tasks.

Why was my command rejected?

Syntax errors display `Error: MESSAGE` followed by `Expected: USAGE`; correct the stated format and retry at the next prompt. Use lowercase commands/prefixes, the specified field order, real dates and existing IDs with the proper prefix. Missing, repeated, reordered or extra fields are rejected. Use `task delete T1`, not `task delete 1` or `task delete T01`. Numeric overflow and leading-zero tokens are rejected rather than silently adjusted.

Task values, unknown IDs and save failures normally produce one `Error:` line without `Expected:`. Task duration `45` is valid but `045` and zero are not; commitments instead require multiples of 30 and half-hour starts. A well-formed new deadline can still fail if it is not future/in range. Rejected commands preserve saved data and the plan; no failed addition consumes an ID. For storage errors, resolve the file/folder problem before retrying. See command format.

What should I do if the application cannot load my data?

Close the app and copy the reported file somewhere safe before attempting a repair. Check the launch folder, access permissions and JAR compatibility. Restore a known-good backup if available, keeping a copy of the failing file for investigation. A malformed/unreadable file is a startup error; the app does not overwrite it with an empty store or silently discard invalid rows. If you choose to start afresh, move the failing file aside first and retain it.

Optional storage diagnostics record processing stages in a separate log file. Storage stage records omit task names and saved contents; broader application errors can contain paths or other details, so inspect any logs before sharing them. Logging does not repair the file. Normal launches stay quiet.

Command summary

These are the supported console commands in this build. Follow an action link for its rules and examples. See command format for optional fields, alternatives, dates, times and IDs.

Action	Command
View help	`help`
Add a task	`task add n/NAME due/YYYY-MM-DD HHmm d/MINUTES`
List tasks	`task list`
Edit a task	`task edit TNUMBER [n/NAME] [due/YYYY-MM-DD HHmm]`
Revise a task's remaining time	`task remaining TNUMBER remaining/MINUTES`
Record completed minutes	`task progress TNUMBER done/MINUTES`
Record session progress	`task progress TNUMBER session/MINUTES percent/PERCENT`
Complete a task	`task complete TNUMBER`
View completed tasks	`task list completed`

Action	Command
Reopen a completed task	`task reopen TNUMBER remaining/MINUTES`
Delete a task	`task delete TNUMBER`
Add a weekly task template	`task recur add n/NAME due/YYYY-MM-DD HHmm d/MINUTES every/WEEKLY [until/YYYY-MM-DD]`
List task templates	`task recur list`
Stop future task generation	`task recur delete RNUMBER`
Add a weekly commitment	`commitment add n/NAME day/DAY start/HHmm d/MINUTES [from/YYYY-MM-DD until/YYYY-MM-DD]`
Add a one-time commitment	`commitment add n/NAME on/YYYY-MM-DD start/HHmm d/MINUTES`
Edit a commitment	`commitment edit CNUMBER [n/NAME] [day/DAY|on/YYYY-MM-DD] [start/HHmm] [d/MINUTES] [from/YYYY-MM-DD until/YYYY-MM-DD]`
Skip one commitment occurrence	`commitment skip CNUMBER on/YYYY-MM-DD`
List commitments	`commitment list`
Delete a commitment	`commitment delete CNUMBER`
Block a continuous interval	`unavailable continuous n/NAME from/YYYY-MM-DD HHmm until/YYYY-MM-DD HHmm`
Block a daily window across dates	`unavailable daily n/NAME from/YYYY-MM-DD until/YYYY-MM-DD start/HHmm end/HHmm`
Set study hours	`hours DAY|YYYY-MM-DD /start HH:mm /end HH:mm`
View effective study hours	`hours DAY|YYYY-MM-DD`
Reset study hours	`hours reset DAY|YYYY-MM-DD`
Set study policy	`policy set max/MINUTES break/MINUTES`
View study policy	`policy show`
Reset study policy	`policy reset`
Generate or replace the plan	`plan`
View today's schedule	`schedule today`
View a dated schedule	`schedule YYYY-MM-DD`
Exit	`exit`

`TNUMBER`, `CNUMBER` and `RNUMBER` mean existing IDs such as `T1`, `C1` and `R1`. For `DAY`, use uppercase three-letter codes such as `MON` for commitments and full lowercase names such as `monday` for study hours. `WEEKLY` is a literal keyword. `PERCENT` is a whole number from `0` to `100`, representing the completed percentage of the stated session. Replace other placeholders with your values.

For commitment edits, supply at least one field; choose `day/` or `on/` if changing when it occurs. `on/` cannot be combined with `from/` or `until/`. Unavailable commands validate and save every affected date together; see blocking multiple dates.

Back to top